Software Design Analysis Report

A. Project Details

Project title: Municipal Complaint Management System (MCMS)

Team / roll numbers: Aastha Sheoran (2400290120004), Ananya Rajput (2400290120042), Aashi Chaudhary (2400290120003), Aeishikka Tyagi (2400290120023)

Project objective (2–3 lines): MCMS is a web-based platform where citizens submit and track municipal complaints, and officers, field workers and administrators manage the complaint lifecycle: verification, assignment, resolution, feedback and reporting. AI is used only as an advisory layer for classification, priority and duplicate detection.

Technology stack: React.js, CSS / Tailwind CSS, Node.js, Express.js, MongoDB, JWT, bcrypt, Multer, Cloudinary, Leaflet + OpenStreetMap, Recharts

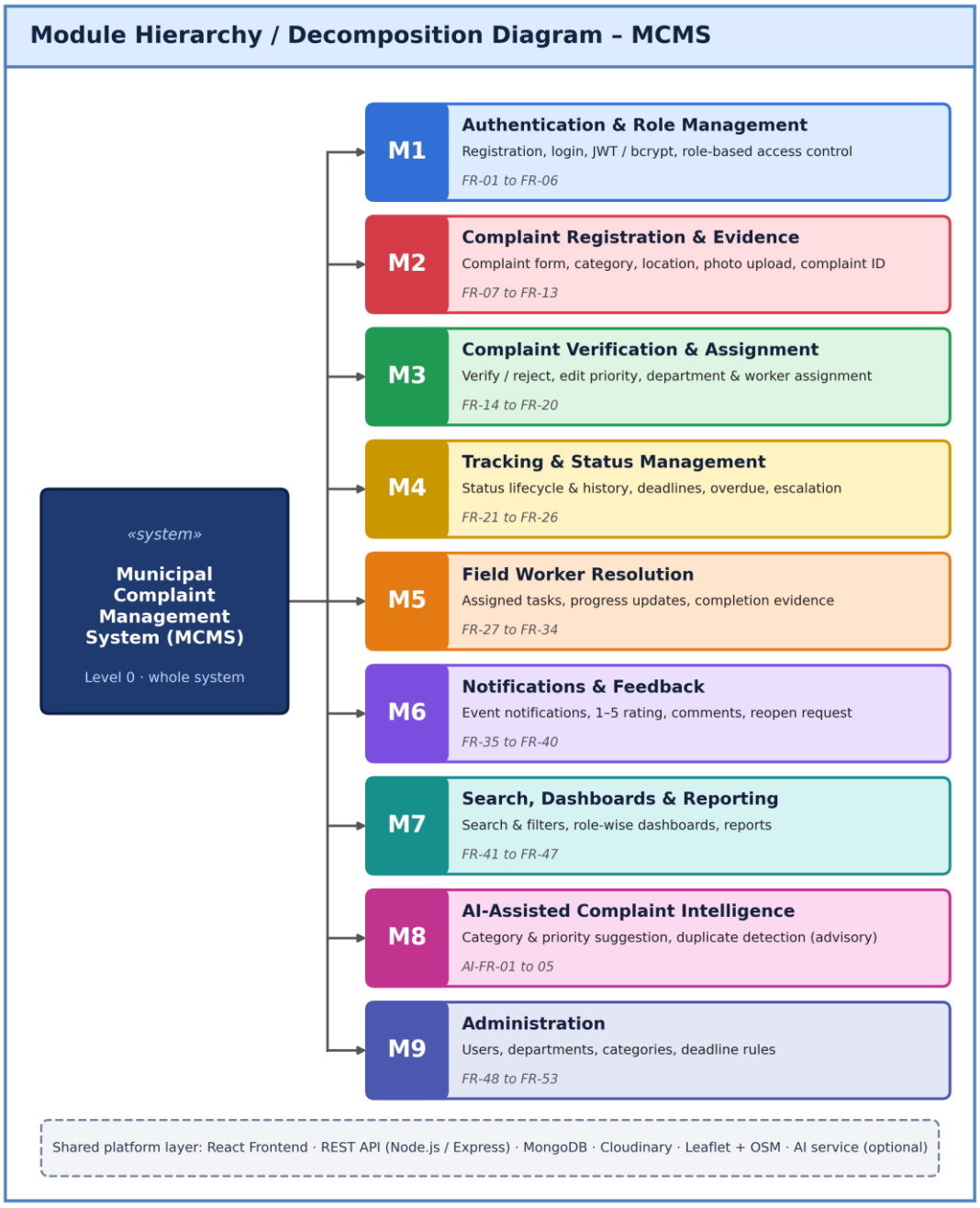
B. Module Identification and Modularization

Total number of modules: 9. MCMS is divided into 9 modules, one for each of the 9 system features in SRS Section 4 (4.1 to 4.9). Every module has its own group of functional requirements and its own user flow, which keeps modules independent and supports the modular architecture required by SRS 5.4 (Maintainability). The AI module (M8) is kept separate because the core system must keep working when AI services are unavailable (SRS 1.4, AI-FR-05).

Module ID	Module name	Main responsibility	Inputs / outputs	Depends on
M1	Authentication & Role Management	Registration, login, JWT / bcrypt security, role-based access control (FR-01 to FR-06)	In: name, email, phone, address, password Out: JWT token, role-based dashboard	None (MongoDB only)
M2	Complaint Registration & Evidence	Complaint form, category, location, photo upload, unique complaint ID (FR-07 to FR-13)	In: title, description, category, location, photo Out: complaint record, complaint ID	M1, M9, M8 (optional)
M3	Complaint Verification & Assignment	Officer verify / reject, edit category and priority, department and worker assignment (FR-14 to FR-20)	In: officer decision, rejection reason, department, worker, deadline Out: Verified / Rejected status, assignment record	M1, M2
M4	Tracking & Status Management	Status lifecycle, status history, deadlines, overdue, escalation (FR-21 to FR-26)	In: status change, actor, deadline Out: current status, history timeline, overdue flag	M2, M3
M5	Field Worker Resolution	Assigned tasks, progress updates, completion evidence, resolution approval (FR-27 to FR-34)	In: progress, before / after images Out: updated status, evidence, resolution request	M1, M3, M4
M6	Notifications & Feedback	Event notifications, 1–5 rating, comments, reopen request (FR-35 to FR-40)	In: complaint event, rating, comment Out: notification, feedback record, reopen request	M4
M7	Search, Dashboards & Reporting	Search, filters, role-wise dashboards, reports (FR-41 to FR-47)	In: filters, report type Out: complaint list, statistics, reports	M2, M4, M5, M6

M8	AI-Assisted Complaint Intelligence	Category suggestion, priority suggestion, duplicate detection (AI-FR-01 to AI-FR-05)	In: complaint text and details Out: suggested category, priority, duplicate flag	None (receives text from M2)
M9	Administration	Users, departments, categories, deadline rules (FR-48 to FR-53)	In: user, department, category, rule data Out: updated master records	M1

Module hierarchy / decomposition diagram:

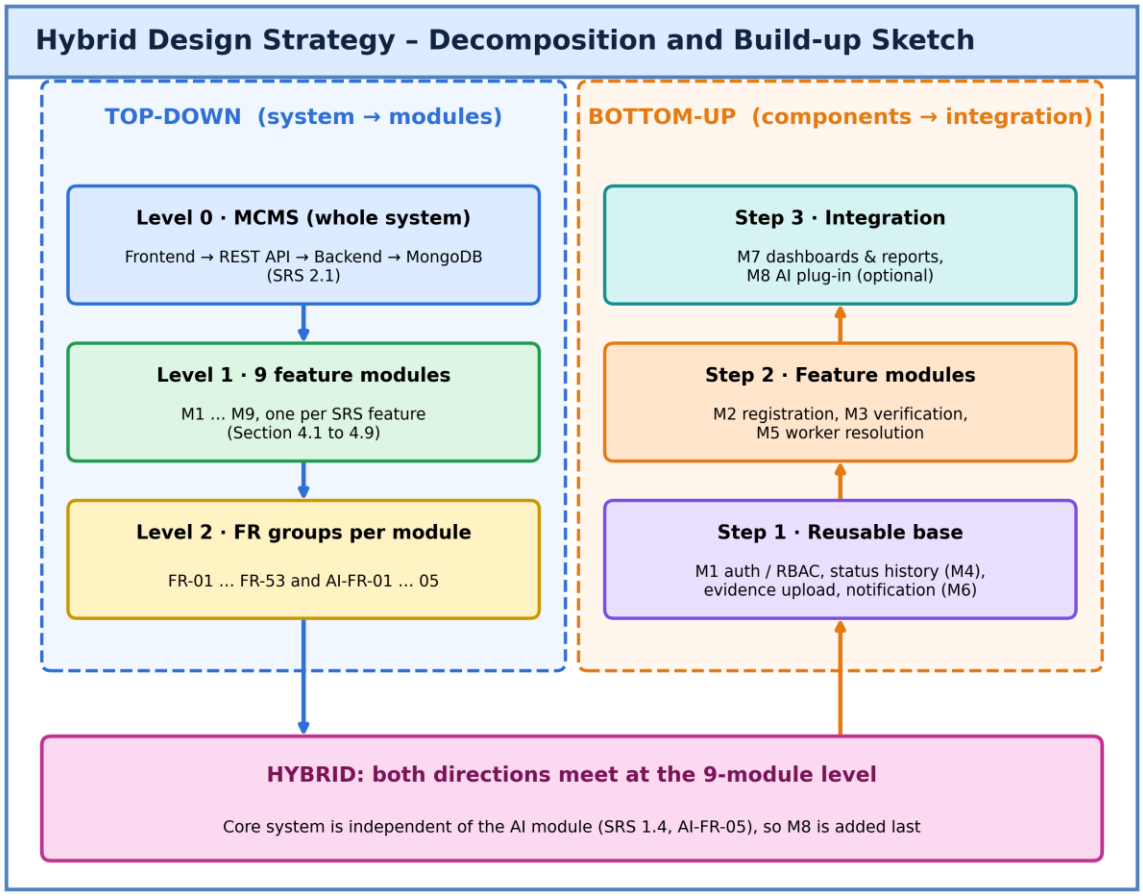


C. Design Strategy

Selected strategy: ☐ Top-Down ☐ Bottom-Up ☒ Hybrid

Justification (4–6 lines): MCMS was first viewed as one complete system (Frontend, REST API, Backend, MongoDB; SRS 2.1) and then decomposed into 9 feature modules (top-down). Some components are used by almost every

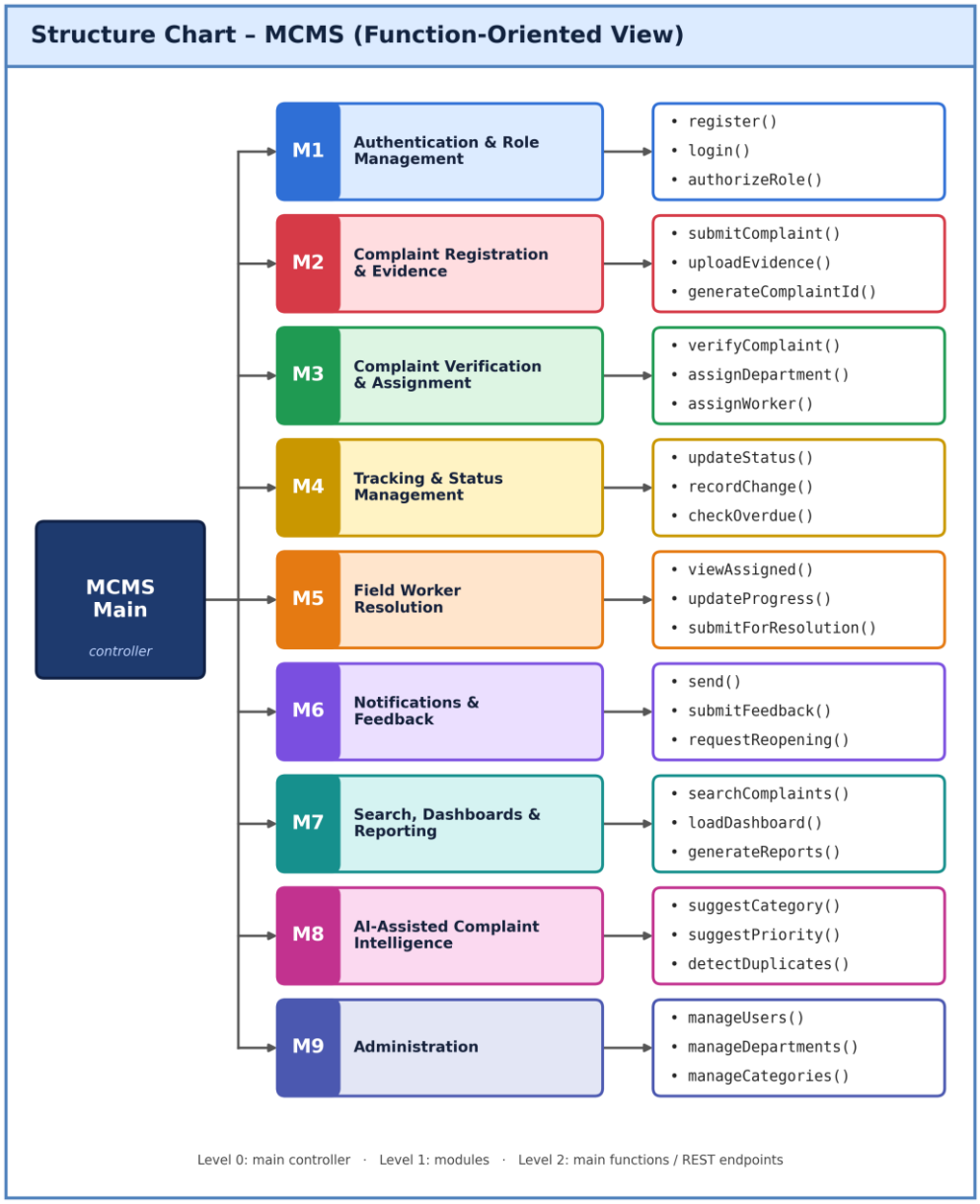
module: authentication and role check (M1), status history and notification (M4, M6) and evidence upload (M2, M5). These had to be designed first as reusable parts (bottom-up). The AI module is independent of the core system (SRS 1.4, AI-FR-05) and is plugged in later. Neither a purely top-down nor a purely bottom-up strategy fits, so a Hybrid strategy is used.
Evidence: Top-down: MCMS, then 9 modules, then their FR groups. Bottom-up: reusable base (M1 authentication, status history, evidence upload, notification), then feature modules (M2, M3, M5), then integration (M7 dashboards and reports).



D. Design Approach

Selected approach: ☐ Function-Oriented ☐ Object-Oriented ☒ Mixed

Justification (4–6 lines): The domain is modelled with classes (class diagram): User (Citizen, MunicipalOfficer, FieldWorker, Administrator), Complaint, Assignment, Evidence, StatusHistory, Feedback, Notification, Department and Category. Data and behaviour belong together here, so Object-Oriented fits. The complaint lifecycle (submit, verify, assign, resolve, close) is a sequence of steps, and each step is a function / REST endpoint, so a Function-Oriented structure chart fits. Hence a Mixed approach: object-oriented for the domain, function-oriented for the workflow.
Evidence: Structure chart: MCMS Main at the top, the 9 modules below it, and the main functions of each module beneath them (for example submitComplaint(), verifyComplaint(), assignWorker(), updateStatus(), uploadEvidence(), send()), based on the functional requirements and the class diagram methods.



E. Coupling Analysis

For at least three important module-to-module interactions, identify the information exchanged, dependency and coupling type (where applicable). Aim for low coupling.

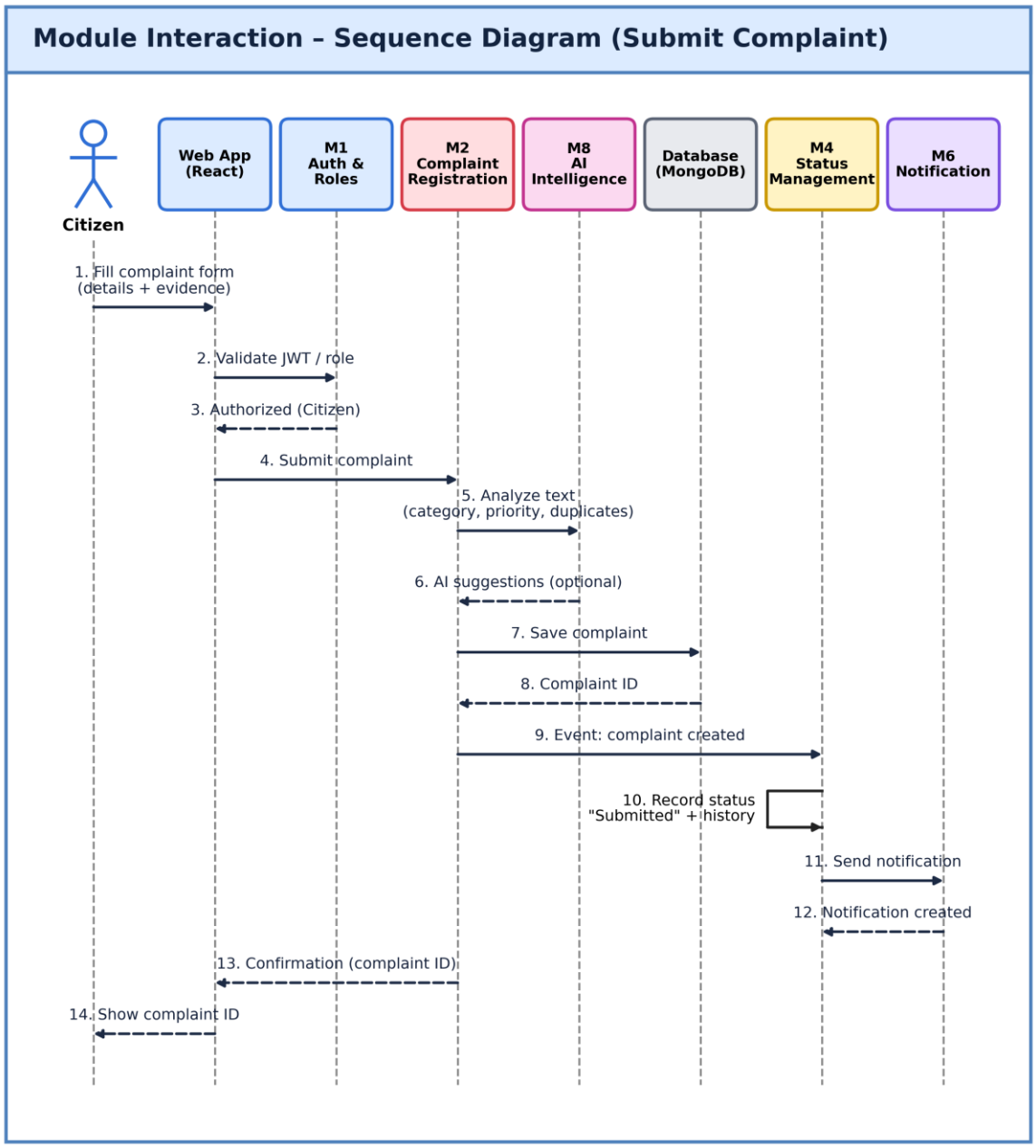
Module pair	Interaction / shared data	Coupling type / level	How to reduce dependency
M2 → M8	Complaint text is sent; suggested category, priority and duplicate flag are returned	Data coupling (low)	Pass only text / ID, not the whole object. Keep a manual fallback (AI-FR-05) so M2 works when AI is down.
M3 → M4	Status change on verify / assign, with actor and timestamp (FR-23)	Data coupling (low)	Use one status-update interface; M3 never writes directly to the status history of M4.
M5 → M3	Worker reads own assignment (BR-06): complaint ID, worker ID, deadline	Data / stamp coupling (low to medium)	Pass only the required fields instead of the full Assignment object.
M4 → M6	Status event creates a notification (FR-35)	Data coupling (low)	Use an event-based trigger so M4 does not depend on notification internals.

F. Cohesion Analysis

For at least three modules, describe the single main responsibility and evaluate how closely its internal tasks belong together. Aim for high cohesion.

Module	Main responsibility	Cohesion type / level	Improvement, if needed
M1 Authentication	Verify identity and control access	Functional (highest)	None needed
M2 Complaint Registration	Store a valid complaint record (form, location, evidence, ID)	Sequential (high)	Move evidence validation into a separate helper function.
M8 AI Intelligence	Give suggestions from complaint text	Functional (high)	None needed
M6 Notifications & Feedback	Inform users of complaint events and collect feedback	Communicational (medium)	Split notification and feedback / reopen into separate sub-components, since their tasks and users differ.

G. Module Interaction and Design Quality



Flow: Citizen logs in through M1 (token), submits the complaint to M2, M2 requests optional AI suggestions from M8, the complaint is saved in the database and a complaint ID is returned, M4 records the status 'Submitted' with history, M6 sends the notification, and the citizen receives the confirmation. The officer then continues with M3 (verify / assign).